

Escuela Politécnica Nacional

Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: EJERCICIO 14

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Implementación y Código Fuente

El código fuente se estructuró agrupando las expresiones regulares mediante el operador lógico alternativo (`()`). Esto permitió asociar una única acción en C para múltiples variaciones de un mismo tipo de operador. Adicionalmente, se incorporaron reglas de exclusión (`[ \t\n]+` y `.`) para manejar el descarte silencioso de cualquier carácter que no corresponda a los operadores requeridos, evitando el comportamiento de impresión por defecto de Lex.

- `"+"|"-"|"*"|"/"`: Agrupación de operadores aritméticos mediante el operador OR léxico.
- `"<="|">="|"=="|"!="|"<"|">"`: Agrupación de operadores relacionales lógicos.
- `[ \t\n]+ { }`: Absorbe espacios en blanco, tabulaciones y saltos de línea sin ejecutar ninguna acción en C (filtro de formato).
- `. { }`: Regla comodín final que absorbe cualquier otro carácter que no haya hecho *match* arriba (filtro de ruido).

Tipos de entradas que recibe: Operadores aritméticos simples y relacionales compuestos o simples inmersos dentro de sentencias matemáticas o lógicas complejas.

Cadenas con las que el código fallaría (Errores Léxicos):

- Cadena `> =` (con espacio): El autómata fallará al intentar reconocerlo como un operador "mayor o igual que". Leerá el `>`, ignorará el espacio por la regla de exclusión, y leerá el `=` (el cual descartará al no estar en sus operadores).
- Cadena `++` o `+=`: Operadores compuestos no declarados. El programa leerá el primer `+` como operador aritmético y procesará el segundo símbolo de forma aislada, dividiendo incorrectamente el operador de incremento.

Código fuente del archivo `catorce.l`:

```
fmerino@fmerino: ~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/CATORCE
GNU nano 8.4 catorce.l
%{
#include <stdio.h>
%}

%%
"+"|"-"|"*"|"/"          { printf("OPERADOR ARITMETICO \"%s\"\n", yytext); }
"<="|">="|"=="|"!="|"<|">" { printf("OPERADOR RELACIONAL \"%s\"\n", yytext); }
[ \t\n]+                  { /* Ignora espacios en blanco, tabuladores y saltos de linea */ }
.                          { /* Ignora cualquier otro caracter no reconocido */ }
%%

int main()
{
    yylex();
    return 0;
}
```

Archivo entrada.txt:

```
fmerino@fmerino: ~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/CATORCE
GNU nano 8.4 entrada.txt
a + b = c
x >= 10
y != 5
contador = contador + 1
valor * 2 <= limite / 3
```

2. Capturas de Ejecución

Se procesó un archivo de texto con múltiples sentencias que combinaban variables, asignaciones, números y los operadores a evaluar.

Ejecución final: Prueba de Procesamiento y Redirección (Consola y Archivo salida.txt):

```
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM$ cd CATORCE
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/CATORCE$ touch catorce.l entrada.txt
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/CATORCE$ nano catorce.l
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/CATORCE$ lex catorce.l
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/CATORCE$ nano entrada.txt
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/CATORCE$ gcc lex.yy.c -lfl
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/CATORCE$ ./a.out < entrada.txt > salida.txt
fmerino@fmerino:~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/CATORCE$ cat salida.txt
OPERADOR ARITMETICO "+"
OPERADOR RELACIONAL ">="
OPERADOR RELACIONAL "!="
OPERADOR ARITMETICO "+"
OPERADOR ARITMETICO "*"
OPERADOR RELACIONAL "<="
OPERADOR ARITMETICO "/"
```

Descripción de lo más relevante de lo observado:

- **Supresión del ruido léxico:** Al aplicar la regla del comodín (.), el programa ignoró sistemáticamente las letras y números (ej. a, b, 10, contador), demostrando que es posible construir analizadores restrictivos que busquen únicamente tokens de interés particular sin generar salidas residuales en la consola.

- **Resolución de prefijos comunes:** El analizador diferenció sin fallos los operadores simples (como <) de los compuestos (como <=). Esto ocurre porque Lex aplica el principio del emparejamiento más largo; al leer el carácter <, examina el siguiente símbolo y, si es un =, prioriza la regla del operador compuesto.

3. Conclusión

La implementación de reglas de exclusión explícitas es una práctica fundamental para controlar la salida del escáner. Agrupar expresiones con el operador lógico alternativo simplifica el código, mientras que aislar únicamente los tokens de interés purga el flujo de entrada de elementos sin peso semántico, preparando un terreno limpio para el analizador sintáctico.